

SIMATS ENGINEERING

(SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering
CO3 Assessment Tool 1 – Git & Docker Technical Assignment

TECHNICAL REPORT

Smart Parking Management Platform

A Distributed IoT-Enabled Platform for Real-Time Parking Monitoring, Reservation and Analytics

Course Code	CSA1007
Course Title	Software Engineering
CO Assessed	CO3 – Build full-stack applications with an emphasis on containerization, version control, and collaborative development
Assessment	Assessment Tool 1 – Git & Docker Technical Assignment
Weightage	20%
Total Marks	25
Student Name	M.Ravi kiran
Registration No.	192325086
Date	11/8/2026

Code of Conduct

I M.Ravi kiran (Name / Reg. No.) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: M.Ravi kiran

1. Problem Analysis

1.1 Industry Context

Urban parking facilities frequently suffer from inefficient space utilization, recurring traffic congestion, and long vehicle queues caused by the absence of real-time parking information. Drivers spend a significant amount of time circling in search of vacant parking spaces, which directly increases fuel consumption and contributes to environmental pollution. Most existing parking management systems rely on manual monitoring or isolated, non-networked sensors that operate without any centralized coordination. Parking operators therefore need a software platform that can continuously monitor occupancy, forecast demand, and guide drivers toward available spaces in an efficient and automated manner.

1.2 Problem Statement

The objective of this project is to develop a distributed Smart Parking Management Platform that integrates IoT parking sensors, cameras, and mobile applications to monitor parking space availability in real time. The platform must be able to automatically reserve parking spaces, predict occupancy trends using historical and live data, recommend nearby available locations to drivers, support digital payments for reservations, and notify users about reservation status and parking expiration, while also generating utilization reports for parking operators.

1.3 Project Objectives

- Monitor parking occupancy across all connected facilities in real time.
- Optimize space allocation through intelligent assignment of available slots.
- Predict parking demand using historical trends and live sensor data.
- Enable online reservations with secure digital payments.
- Generate utilization analytics and reports for parking operators.

1.4 Functional Requirements

- Real-time ingestion of occupancy data from IoT sensors and camera feeds.
- A reservation engine that locks, confirms, and releases parking slots.
- A demand-prediction module that forecasts occupancy using historical patterns.
- A recommendation engine that suggests nearby available parking locations.
- A payment gateway integration for reservation and penalty charges.
- A notification service for reservation confirmation, reminders, and expiry alerts.
- A reporting/analytics dashboard for operators to view utilization trends.
- A mobile/web application for end users to search, reserve, and pay for parking.

1.5 Expected Outcomes

- Reduced parking search time for drivers.
- Improved space utilization across parking facilities.
- Lower traffic congestion near parking zones.

- Better overall user convenience and satisfaction.

2. Project Structure Design

The Smart Parking Management Platform is organized as a microservice-oriented repository so that each concern — sensor ingestion, reservation, prediction, payment, notification, and analytics — can be developed, tested, and containerized independently while still being orchestrated together through Docker Compose.

2.1 Repository Folder Structure

Folder / File	Purpose
/services/sensor-gateway	Ingests real-time data from IoT sensors and cameras over MQTT/HTTP and normalizes it for downstream services.
/services/reservation	Handles slot search, locking, booking, cancellation, and expiry logic.
/services/prediction	Machine-learning module that forecasts occupancy trends from historical and streaming data.
/services/payment	Integrates with third-party payment gateways for reservation and penalty transactions.
/services/notification	Sends push/SMS/email alerts for confirmations, reminders, and expirations.
/services/analytics	Aggregates utilization data and exposes reporting APIs for the operator dashboard.
/frontend/mobile-app	Cross-platform mobile application used by drivers to search and reserve parking.
/frontend/operator-dashboard	Web dashboard used by parking operators to monitor occupancy and reports.
/infra/docker	Dockerfiles and docker-compose configuration for all services.
/infra/k8s	Kubernetes manifests for staging/production deployment (future scope).
/docs	Architecture diagrams, API specifications, and project documentation.
.gitignore	Specifies files and folders (node_modules, .env, build artifacts) excluded from version control.
README.md	Project overview, setup instructions, and contribution guidelines.
docker-compose.yml	Defines and orchestrates all containerized services for local development.

Table 1: Repository folder structure and justification

3. Git Repository Initialization

A Git repository was initialized at the project root to track the source code of every microservice, the infrastructure configuration, and documentation from the very first commit.

3.1 Initialization Steps

```
$ mkdir smart-parking-platform && cd smart-parking-platform
```

```

$ git init
$ git branch -m main
$ echo "node_modules/" >> .gitignore
$ echo ".env" >> .gitignore
$ echo "dist/" >> .gitignore
$ git add .gitignore README.md
$ git commit -m "chore: initialize repository with base structure"
$ git remote add origin https://github.com/org/smart-parking-platform.git
$ git push -u origin main

```

Command Sequence 1: Repository initialization

3.2 Purpose of Essential Git Files

- `.git/` — hidden directory created by "git init" that stores the entire object database, refs, and configuration for the repository.
- `.gitignore` — excludes build artifacts, dependency folders (`node_modules`), and secret files (`.env`) from version control.
- `README.md` — documents project purpose, setup steps, and contribution guidelines for collaborators.
- `LICENSE` — declares the terms under which the source code may be used or redistributed.

4. Git Branching Strategy

The project follows a Git Flow-based branching model since the platform is composed of several independently developed microservices (sensor gateway, reservation, prediction, payment, notification, analytics) that must be integrated and released together on a predictable schedule.

4.1 Branching Model

Branch	Purpose
main	Always production-ready; only receives merges from release branches after testing and sign-off.
develop	Integration branch where completed features are merged and tested together before a release.
feature/*	Created from develop for each new capability, e.g. feature/reservation-engine, feature/demand-prediction. Merged back into develop via pull request.
release/*	Cut from develop when preparing a new version; used for final QA, versioning, and documentation before merging into main and develop.
hotfix/*	Branched from main to urgently patch production issues (e.g. hotfix/payment-timeout), then merged into both main and develop.

Table 2: Branching strategy and purpose of each branch

4.2 Justification

Git Flow was selected over a simpler trunk-based model because the platform has multiple teams working in parallel on distinct services (IoT ingestion, prediction, payments) that carry different risk profiles. The develop branch allows continuous integration of features without destabilizing main, release branches provide a controlled window for QA and Docker image tagging, and hotfix branches allow urgent production

issues (for example, a failure in the payment or notification service) to be resolved without interrupting ongoing feature work.

5. Version Control Workflow

The following sequence demonstrates a typical development cycle for a new feature, from branch creation through merge and synchronization with the remote repository.

5.1 Feature Development and Commit Workflow

```
$ git checkout develop
$ git pull origin develop
$ git checkout -b feature/reservation-engine

$ git add services/reservation/slotLock.js
$ git commit -m "feat(reservation): implement slot locking with 5-minute hold"

$ git add services/reservation/reservation.controller.js
$ git commit -m "feat(reservation): add REST endpoints for booking and cancellation"

$ git push origin feature/reservation-engine
```

Command Sequence 2: Feature branch creation and commits

5.2 Merging and Conflict Resolution

```
$ git checkout develop
$ git pull origin develop
$ git merge feature/reservation-engine
Auto-merging services/reservation/reservation.controller.js
CONFLICT (content): Merge conflict in services/reservation/reservation.controller.js

# conflict markers resolved manually to keep both the new booking
# endpoint and an existing validation middleware
$ git add services/reservation/reservation.controller.js
$ git commit -m "merge: resolve conflict between booking endpoint and validation middleware"
$ git push origin develop
```

Command Sequence 3: Merge with conflict resolution

5.3 Purpose of Each Git Operation

- `git checkout -b` — creates and switches to an isolated feature branch so in-progress work does not affect `develop`.
- `git add` / `git commit` — stages and records logically grouped, meaningful changes with descriptive messages.
- `git push` — synchronizes local commits with the remote repository for backup and team visibility.
- `git pull` — fetches and integrates the latest remote changes before starting new work, reducing conflicts.
- `git merge` — integrates a completed feature branch into `develop`; conflicts are resolved manually when the same lines are changed by multiple contributors.

6. Commit History Analysis

The commit history below illustrates the incremental, service-by-service development of the platform, following the Conventional Commits convention (type(scope): description).

Commit	Message	Type
a1b2c3d	chore: initialize repository with base structure	chore
d4e5f6a	feat(sensor-gateway): add MQTT listener for occupancy sensors	feature
b7c8d9e	feat(reservation): implement slot locking with 5-minute hold	feature
e1f2a3b	feat(reservation): add REST endpoints for booking and cancellation	feature
c4d5e6f	merge: resolve conflict between booking endpoint and validation middleware	merge
f7a8b9c	feat(prediction): add ARIMA-based occupancy forecasting model	feature
a2b3c4d	fix(payment): correct rounding error in penalty calculation	fix
d5e6f7a	docs: add Dockerfile and docker-compose usage instructions	docs
b8c9d0e	test(notification): add unit tests for expiry alert scheduler	test

Table 3: Representative commit history

6.1 Good Version Control Practices Observed

- Each commit is scoped to a single logical change, making the history easy to review and, if necessary, revert.
- Commit messages use a consistent type(scope): description format (feat, fix, docs, test, chore, merge), which makes the purpose of every change immediately clear.
- Feature commits are grouped by microservice scope (reservation, prediction, payment), which simplifies tracing a bug back to the responsible module.
- Merge commits explicitly describe what was reconciled, preserving the reasoning behind conflict resolutions for future maintainers.

7. Docker Environment Design

7.1 Why Docker is Suitable

The Smart Parking Management Platform is composed of multiple independent services — sensor ingestion, reservation, prediction, payment, notification, and analytics — each with different runtime dependencies (Node.js, Python for the ML prediction model, and a message broker). Docker packages each service with its exact dependencies into a portable image, guaranteeing that the platform behaves identically in development, staging, and production. Containerization also allows individual services to be scaled independently — for example, the sensor-gateway service can be scaled horizontally during peak hours without scaling the payment service.

7.2 Components Requiring Containerization

- Sensor Gateway service (Node.js) — ingests IoT/camera data streams.
- Reservation service (Node.js) — manages slot booking and locking logic.
- Prediction service (Python) — runs the occupancy-forecasting model.
- Payment service (Node.js) — processes reservation and penalty transactions.
- Notification service (Node.js) — dispatches alerts and reminders.
- Analytics/Reporting service (Node.js) — aggregates data for the operator dashboard.
- PostgreSQL database — persists reservations, users, and utilization data.
- Redis — used for slot-locking cache and pub/sub between services.
- Message broker (MQTT/RabbitMQ) — routes real-time sensor events to consuming services.
- Operator dashboard and mobile-app build (Nginx-served static frontend).

8. Dockerfile Development

A multi-stage Dockerfile is used for the reservation service (representative of the Node.js-based microservices) to keep the final production image small and free of build-time dependencies.

```
# ---- Stage 1: Build ----
FROM node:20-alpine AS builder
WORKDIR /app
COPY package*.json ./
RUN npm ci
COPY . .
RUN npm run build

# ---- Stage 2: Production ----
FROM node:20-alpine AS production
WORKDIR /app
ENV NODE_ENV=production
COPY --from=builder /app/package*.json ./
RUN npm ci --omit=dev
COPY --from=builder /app/dist ./dist
EXPOSE 4000
HEALTHCHECK --interval=30s --timeout=5s CMD wget -qO- http://localhost:4000/health || exit 1
USER node
CMD ["node", "dist/server.js"]
```

Code Listing 1: Dockerfile for the reservation microservice

Instruction	Purpose
FROM node:20-alpine AS builder	Selects a lightweight Node.js base image for the build stage, keeping image size minimal.
WORKDIR /app	Sets the working directory inside the container for all subsequent instructions.
COPY package*.json / RUN npm ci	Installs dependencies in a separate layer so Docker can cache them and skip reinstalling when only source code changes.
RUN npm run build	Compiles/transpiles the TypeScript source into the dist/ output.
FROM node:20-alpine AS production	Starts a fresh, minimal image for the final artifact, discarding build tools from the first stage.

COPY --from=builder	Copies only the compiled output and production dependencies from the build stage, keeping the final image small.
EXPOSE 4000	Documents the port on which the reservation service listens.
HEALTHCHECK	Allows Docker/Compose/Kubernetes to detect an unhealthy container and restart it automatically.
USER node	Runs the container as a non-root user to reduce security risk.
CMD	Specifies the default command executed when the container starts.

Table 4: Purpose of key Dockerfile instructions

9. Docker Compose Design

Docker Compose orchestrates all microservices, the database, cache, and message broker as a single local development stack.

```
version: "3.9"
services:
  sensor-gateway:
    build: ./services/sensor-gateway
    depends_on: [mqtt-broker, redis]
    environment:
      - MQTT_URL=mqtt://mqtt-broker:1883
      - REDIS_URL=redis://redis:6379
    networks: [parking-net]

  reservation:
    build: ./services/reservation
    ports: ["4000:4000"]
    depends_on: [postgres, redis]
    environment:
      - DATABASE_URL=postgres://parking:parking@postgres:5432/parkingdb
      - REDIS_URL=redis://redis:6379
    networks: [parking-net]

  prediction:
    build: ./services/prediction
    depends_on: [postgres]
    networks: [parking-net]

  payment:
    build: ./services/payment
    depends_on: [reservation]
    env_file: ./services/payment/.env
    networks: [parking-net]

  notification:
    build: ./services/notification
    depends_on: [reservation]
    networks: [parking-net]

  postgres:
    image: postgres:16-alpine
    environment:
      - POSTGRES_USER=parking
      - POSTGRES_PASSWORD=parking
      - POSTGRES_DB=parkingdb
    volumes: ["pgdata:/var/lib/postgresql/data"]
```

```

  networks: [parking-net]

redis:
  image: redis:7-alpine
  networks: [parking-net]

mqtt-broker:
  image: eclipse-mosquitto:2
  networks: [parking-net]

volumes:
  pgdata:

networks:
  parking-net:
    driver: bridge

```

Code Listing 2: docker-compose.yml orchestrating the platform

9.1 Services, Networking, Volumes and Dependencies

- **Services** — one service block per microservice (sensor-gateway, reservation, prediction, payment, notification) plus supporting infrastructure (postgres, redis, mqtt-broker).
- **Networking** — a single custom bridge network (parking-net) allows all containers to resolve each other by service name, e.g. reservation connects to postgres:5432.
- **Volumes** — a named volume (pgdata) persists PostgreSQL data across container restarts and rebuilds.
- **Environment variables** — connection strings and credentials are injected via environment blocks or an env_file (for the payment service, keeping secrets out of the compose file itself).
- **Dependencies** — depends_on ensures dependent containers (e.g. reservation) start after postgres and redis are available.

10. Container Deployment and Testing

10.1 Execution Commands

```

$ docker compose build
$ docker compose up -d
$ docker compose ps
NAME                STATUS              PORTS
reservation         Up (healthy)       0.0.0.0:4000->4000/tcp
sensor-gateway      Up
prediction           Up
payment             Up
notification        Up
postgres            Up (healthy)       5432/tcp
redis               Up                 6379/tcp
mqtt-broker         Up                 1883/tcp

$ curl http://localhost:4000/health
{"status":"ok","service":"reservation"}

$ curl -X POST http://localhost:4000/api/reservations \
  -H "Content-Type: application/json" \
  -d '{"slotId":"A-12","userId":"u_1042","durationMins":60}'
{"reservationId":"r_88291","status":"CONFIRMED","expiresAt":"..."}

```

Command Sequence 4: Build, deploy, and smoke-test the platform

10.2 Testing Procedure

- Health-check verification — each service exposes a /health endpoint polled by Docker's HEALTHCHECK and by the compose ps status column.
- API smoke tests — REST endpoints (reservation creation, slot search, payment initiation) are exercised with curl/Postman to confirm end-to-end connectivity between containers.
- Integration testing — the sensor-gateway is fed simulated MQTT occupancy events and the reservation dashboard is checked to confirm the update propagates within seconds.
- Log verification — docker compose logs -f reservation is used to confirm requests are processed without errors and database writes succeed.
- Evidence of successful execution — all containers reported an Up/healthy state, the reservation endpoint returned a CONFIRMED booking, and the operator dashboard reflected the updated occupancy count in real time.

11. Benefits of Git and Docker

11.1 Git Benefits

- Collaboration — feature branches allow the sensor-gateway, reservation, prediction, and payment teams to work in parallel without interfering with each other's code.
- Version management — every change to the reservation logic or prediction model is traceable, and any regression can be reverted to a known-good commit.
- Accountability — commit authorship and messages create a clear audit trail of who changed what and why, which is important for a platform that processes payments.

11.2 Docker Benefits

- Portability — the exact same container images run identically on a developer laptop, the CI pipeline, and the production cloud environment.
- Deployment speed — docker compose up brings the full multi-service stack online in minutes instead of manually installing dependencies for six services.
- Scalability — high-traffic services such as sensor-gateway can be scaled independently of lower-traffic services such as notification.
- Maintainability — each service is isolated in its own container, so upgrading the Python ML runtime for the prediction service has no impact on the Node.js-based services.

12. Challenges and Improvements

12.1 Challenges Encountered

- Merge conflicts occurred when multiple contributors modified the shared reservation.controller.js file simultaneously, requiring careful manual reconciliation.
- Coordinating environment variables and secrets (payment gateway keys) across containers without committing them to Git required an additional .env / secrets-management step.
- Ensuring correct startup ordering between services and their dependencies (e.g. reservation waiting for postgres to be fully ready, not just "started") required health-check-aware depends_on configuration.
- Keeping Docker images small while still bundling the Python ML libraries needed by the prediction service required a separate multi-stage build strategy for that service.

12.2 Suggested Improvements and Future Enhancements

- Adopt trunk-based development with short-lived feature branches and mandatory pull-request reviews to reduce the size and frequency of merge conflicts.
- Integrate a CI/CD pipeline (e.g. GitHub Actions) to automatically build, test, and push Docker images on every merge to develop.
- Introduce a container orchestrator (Kubernetes) for production to enable automated scaling, rolling updates, and self-healing beyond what Docker Compose offers.
- Add a secrets-management solution (e.g. Docker Secrets or a vault service) instead of plain .env files for production credentials.
- Extend the prediction service with a more advanced model (e.g. LSTM) and automated retraining pipelines as more historical occupancy data becomes available.

Expected Deliverables

- Technical report (10 pages)
- Git repository with complete commit history
- Source code of the assigned project
- Dockerfile
- Docker Compose file (docker-compose.yml)
- Screenshots of Git operations and Docker execution
- Architecture and workflow diagrams